

# Verifier-Guided Evaluation of LLM-Generated Network Configurations: A Paired Study with Shared Initial Candidates

Autonomous AI Researcher  
CNSM 2026 Agentic AI Researcher Track

**Abstract**—We preregistered and executed a paired confirmatory study (CAND-2026-001) that measured the marginal effect of a verifier-triggered guarded pipeline (generate → validate → repair,  $\leq 1$  repair) on identical single-shot initial generations for intent→configuration NetOps tasks under shared\_initial\_candidate semantics. Using a deterministic synthetic task generator and a deterministic network verifier, we ran  $N=200$  preregistered tasks with the hosted model declared as gpt-5-mini (model configuration records temperature = null/unset). Provider-call accounting and deterministic reconciliation show 200 shared initial generation calls and 1 repair call (201 model calls total). Baseline configurations passed the deterministic verifier in 199/200 tasks (99.5%); the guarded pipeline passed 200/200 (100%). The paired contingency counts are  $n_{11}=199$ ,  $n_{10}=1$ ,  $n_{01}=0$ ,  $n_{00}=0$ ; paired absolute risk difference (guarded - baseline) = 0.005; exact two-sided McNemar  $p = 1.0$  (exact-binomial convention); paired-bootstrap 95% percentile CI = [0.00, 0.015] (10,000 resamples, seed 1729). The single observed improvement arose from one verifier-triggered repair that corrected a required-sequence ordering violation in an extreme multi-step task. We report preregistration, deterministic task and verifier design, complete execution accounting (timestamps and provider-call audit), paired confirmatory analysis, representative difficulty diagnostics, and bounded limitations grounded in the archived execution bundle.

## I. INTRODUCTION

Generative large language models (LLMs) are being explored as tools to translate high-level human intent into device configurations for network operations (NetOps). Operational correctness in these workflows requires generated configurations to satisfy deterministic invariants (ordering constraints, allowed-field modifications, group-level availability, and syntactic/semantic invariants). Prior prototype systems and benchmarks show feasibility for intent→configuration synthesis, yet practical adoption hinges on integrating verifiers and bounded repair policies and on quantifying their marginal benefits. A key methodological nuance is whether comparisons reuse the same generated candidate across conditions. When identical initial samples are deterministically reused, any paired difference isolates downstream validator-triggered repair rather than generator-side effects (prompting, sampling, or candidate selection).

We report a preregistered paired experiment (CAND-2026-001) that enforces shared\_initial\_candidate semantics so that both baseline and guarded episodes begin from the exact same single initial generation. This design isolates a repair-only estimand: the paired difference in per-task binary acceptance by

a deterministic verifier after permitting at most one automated repair in the guarded arm. The experiment therefore answers a narrow, operationally relevant question: given one sampled candidate per task, how often can a verifier-triggered bounded repair convert an initially failing configuration into one that satisfies the verifier’s deterministic invariants?

## II. RELATED WORK

Empirical and engineering work on LLM-driven NetOps has progressed rapidly. Recent intent→configuration prototypes and benchmarks establish that LLMs can produce syntactically plausible device configurations and that task-structured datasets enable initial benchmarking [39],[86],[129]. Deterministic configuration verifiers and formal-check approaches provide operational oracles appropriate for assessing syntactic and workflow-ordering invariants in configuration synthesis [2]. In adjacent domains, generate→validate→repair pipelines—in program repair and synthesis—demonstrate that executable checks can guide model-produced artifact correction, motivating guarded designs for NetOps but not guaranteeing comparable effects because network invariants include multi-device sequencing and group-availability constraints [4].

Tool-augmented agent work underscores both opportunity and engineering complexity when coupling LLMs to external validators or solvers: careful oracle specification, protocol safety, and artifacted evaluation are required to reason about operational correctness and auditability [6]. Reporting and preregistration guidance for LLM studies emphasize predeclared estimands, frozen analysis plans, and archival traces; we followed those norms when preregistering CAND-2026-001 and archiving execution artifacts [3].

Synthesis. The supplied literature supports three measured points relevant to our design: (i) intent→configuration pipelines and benchmark-style evaluations are active and reproducible research directions [39],[129]; (ii) deterministic verifiers are a natural operational oracle for configuration correctness and workflow-order constraints [2]; and (iii) generate→validate→repair is a promising architectural pattern demonstrated in adjacent domains but requires domain-specific, verifier-integrated empirical evaluation to quantify repair-only effects under paired, shared-candidate semantics [4],[6]. Our paired shared\_initial\_candidate experiment addresses the specific empirical gap of measuring the marginal

repair effect when the identical initial candidate is reused across baseline and guarded conditions.

### III. METHODOLOGY

Preregistration and estimand. The design and analysis plan were preregistered as CAND-2026-001 prior to any model calls. The primary estimand is the paired\_success\_rate\_difference\_guarded\_minus\_baseline: per-task difference in binary 'all-specified-invariants-pass' rates between guarded and baseline, paired by task. Preregistration fixed task\_count = 200, generation\_semantics = shared\_initial\_candidate (one sampled initial generation per task deterministically reused for both conditions), maximum one automated repair call per task, and primary confirmatory test = exact two-sided McNemar (exact binomial-tail convention). The preregistration and execution manifest are archived in the execution bundle.

Deterministic task bank and verifier oracle. Tasks derive from a deterministic task generator (deterministic\_netops\_task\_generator\_v5) and are selected by a fixed index rule recorded in the preregistration. Each task manifest encodes initial\_state, expected\_final\_state, required\_sequence (ordered field changes), allowed\_touched\_fields, and workflow\_policies (final\_group\_constraints, minimum\_active\_in\_groups, must\_be\_down\_for\_fields). The verifier (deterministic\_netops\_validator\_v5) parses generated configurations, enforces allowed\_fields and required\_sequence ordering, evaluates group-level invariants, and returns a boolean full\_intent\_constraint\_validation only when all encoded constraints are satisfied. The verifier emits structured tracer outputs that are the authoritative scoring evidence; those traces are archived and formed the basis of episode scoring.

Shared\_initial\_candidate semantics and sampling notes. The adapter contract ensured shared\_initial\_candidate semantics: for each pair the same single initial generation (one sampled candidate) was provided to baseline and guarded episodes. Consequently any paired difference can only arise from the deterministic verifier and the single permitted repair; it cannot arise from independent first-pass guarded generations, different sampling calls, prompt variants, or multi-sample selection. The model-configuration record declares the requested model as gpt-5-mini and records temperature = null (unset). We explicitly note that configuration temperature = null/unset is not evidence of deterministic sampling and does not imply temperature = 0; nondeterministic hosted-model sampling therefore remains a possible background property even though each pair used a single sampled initial candidate by design.

Sanitizer, repair policy, and provider-call accounting. A deterministic formatting sanitizer ran before verifier parsing. If the verifier reported invariant violations, guarded logic could invoke at most one automated repair call to the hosted model with a repair prompt constructed from the verifier tracer (preregistered). Repair calls were counted against the maximum\_model\_calls budget. Deterministic reconciliation records authoritative stage-level provider counts:

shared\_initial\_generation = 200 and repair = 1, summing to hosted\_model\_call\_event\_count = 201; cache\_miss\_count = 201 and cache\_hit\_count = 0 indicate fresh provider calls for every stage invocation. All provider-call accounting and stage counts are archived in the execution manifest.

Confirmatory statistical procedures. Primary test: exact two-sided McNemar computed from discordant pair counts n10 and n01 using the exact binomial-tail convention implemented in the archived analysis executor. For paired absolute risk difference we used a paired-bootstrap percentile 95% confidence interval with 10,000 resamples and seed 1729; these parameters and seed are recorded in the analysis log. Primary failed-call treatment completed per preregistration (complete\_pair\_primary); sensitivity analyses (preregistered) treat persistent unparsables as failures. The analysis executor and logs are archived and reproduce the numeric results reported below.

### IV. RESULTS

Execution and audit. The run executed under execution\_mode = scientific\_confirmatory and completed both episodes for all 200 preregistered tasks (planned\_episode\_count = 400, completed\_episode\_count = 400, failed\_episode\_count = 0). The run timestamps record start 2026-09-01T16:03:05.491039+00:00 and end 2026-09-01T16:32:13.298536+00:00 (elapsed  $\approx$  29 minutes). Provider-call reconciliation reports hosted\_model\_call\_event\_count = 201 with stage\_counts = {shared\_initial\_generation: 200, repair: 1} and cache\_miss\_count = 201, matching preregistration expectations for shared\_initial\_candidate semantics and a single repair invocation across all pairs.

Primary confirmatory outcomes. The deterministic verifier judged baseline successes = 199 and failures = 1 (baseline success\_rate = 0.995) and guarded successes = 200 and failures = 0 (guarded success\_rate = 1.000). The paired contingency counts are n11 = 199, n10 = 1, n01 = 0, n00 = 0, yielding paired absolute risk difference (guarded - baseline) = 0.005. With  $n = n10 + n01 = 1$  and  $k = n10 = 1$  the archived exact two-sided McNemar p-value (exact-binomial-tail convention) equals  $p = 1.0$ . The paired-bootstrap percentile 95% confidence interval (10,000 resamples, seed 1729) is [0.00, 0.015]. These numeric results are reproduced by the archived paired\_binary\_analysis\_v1 executor when run on the recorded raw results.

Single repair event and diagnostic trace. The guarded pipeline invoked exactly one automated repair during the run. Archived validator traces for the repaired pair indicate the shared initial candidate violated the task's required\_sequence ordering constraint; after applying the bounded repair the verifier trace documents validation\_after.valid = true. Associated task difficulty metadata for the repaired episode indicate an extreme composite pattern (required\_command\_count up to 9, changed\_interface\_count up to 3, difficulty.level = "extreme"); representative tasks in the archive show required\_command\_count values spanning 3–9 and changed\_interface\_count up to 3. The archived tracer

fields record `repair_applied = true` for the repaired episode and provide the repair prompt and repaired configuration as execution evidence that the single guarded improvement resulted from a verifier-triggered repair that enacted the required reordering.

Missingness, exclusions, and consistency checks. No pre-registered exclusions or persistent unparsables were recorded. The analysis missingness summary reports `complete_pairs = 200` and zeros in all other missingness categories. Deterministic reconciliation reports `pair_accounting_consistent = true` and all `deterministic_consistency_checks_passed = true`. No episodes have terminal errors and provider-call audits align with the preregistered execution contract.

Exploratory diagnostics. Representative task manifests include structured difficulty metadata (`changed_interface_count`, `dependency_rule_count`, `required_command_count`). In archived representative tasks `required_command_count` spans 3–9 and `changed_interface_count` reaches 3; the single baseline failure (and the repaired episode) are associated with an extreme composite difficulty pattern. This localized concentration of residual failures on complex, multi-device sequencing tasks is consistent with the preregistered exploratory hypothesis that dependency and ordering complexity concentrate residual verifier-detectable faults.

Quantitative uncertainty and interpretation. The realized baseline success rate ( $199/200 = 99.5\%$ ) places the confirmatory comparison in a sparse-discordant regime: only one discordant pair limits directional evidence as measured by exact McNemar ( $p = 1.0$ ), while the paired-bootstrap CI  $[0.00, 0.015]$  bounds plausible absolute improvement attributable to repair in the repair-only estimand. In operational terms the data are compatible with at most small absolute improvements under the specific repair-only comparison enforced by `shared_initial_candidate` semantics.

## V. DISCUSSION

What this experiment measured. Under `shared_initial_candidate` semantics the study isolates the marginal effect of permitting a single verifier-triggered bounded repair on the identical initial generation. Because baseline and guarded conditions began from the same initial candidate per task, any paired difference is necessarily downstream-of-generation and therefore attributable to the verifier+repair logic. The observed paired outcome (one repaired success) thus provides direct evidence that, for at least some extreme multi-step ordering faults expressible in the task manifest invariants, a bounded verifier-triggered repair can convert a failing configuration into one that satisfies the deterministic verifier.

Statistical regime, ceiling effects, and power. The preregistered power rationale anticipated detecting moderate absolute paired increases ( $\sim 10$ – $15$  percentage points) at  $N = 200$  under mid-range baseline assumptions ( $\sim 45$ – $65\%$ ). The realized baseline success rate ( $99.5\%$ ) created a ceiling effect that substantially reduced realized sensitivity to small repair-only

effects: with only one discordant pair the exact McNemar  $p$ -value conveys limited inferential directionality. The paired-bootstrap CI  $[0.00, 0.015]$  quantifies the uncertainty and indicates the upper bound of plausible absolute improvement attributable to repair under our repair-only estimand given the observed data and resampling scheme (10,000 resamples, seed 1729).

Failure-mode characterization. Archived task-level tracer fields and difficulty metadata show that the remaining verifier-detectable failures concentrate on complex multi-device workflows that encode strict `required_sequence` ordering constraints and multiple dependent field changes. Representative archived tasks with extreme difficulty patterns (`required_command_count = 9`, `changed_interface_count = 3`) show higher fragility under single-shot generation than simpler tasks. The single repaired episode involved a `required_sequence` ordering breach corrected by the bounded repair; the archived repair prompt and post-repair validation trace document the repair logic and its corrective effect. These observations support a practical hypothesis (exploratory) that ordering and staged-rollout constraints—rather than simple per-device syntactic errors—are a dominant residual class of verifiable failures when initial generation quality is otherwise high.

Operational implications and bounded claims. Artifact-grounded evidence supports the narrow operational conclusion that, in this synthetic deterministic task bank evaluated with `gpt-5-mini` and `deterministic_netops_validator_v5`, a verifier-guided bounded repair can correct certain ordering/dependency violations in generated configurations. This is evidence for repair efficacy only in the restricted, verifier-expressive setting studied: it is not evidence that the same repair policy would fix runtime-only dataplane omissions, vendor-runtime semantic mismatches, adversarially poisoned contexts, or that similar gains would obtain under independent guarded first-pass generation semantics or with other LLMs. Those remain open empirical axes requiring multi-model comparisons, independent-first-pass guarded arms, and live-device or dataplane emulation.

Threats to validity. Internal validity is strengthened by deterministic task selection, archived verifier traces, and the paired `shared_initial_candidate` contract, which isolates repair-only effects. However, the `shared_initial_candidate` design does not address prompt sensitivity or sampling variability across multiple independent generator draws. Construct validity: the deterministic verifier enforces encoded invariants but does not emulate full device runtime or vendor-specific runtime behaviors; passing the verifier is necessary for the encoded invariants but not sufficient to guarantee safe production deployment in heterogeneous live environments. External validity: the study used a single hosted model (declared `gpt-5-mini`) and a synthetic deterministic task bank; broader generalization requires further experiments across models, richer task banks, and live emulation.

Practical takeaways for system designers. If initial single-shot generation quality is already high, bounded verifier-

triggered repair under a shared-initial regimen yields at most small marginal gains in per-task verifier acceptance in our synthetic verifier-backed setting; however, the repair mechanism demonstrably corrected an extreme sequencing violation when it occurred. Designers should therefore consider where verification invariants expose correctable ordering or dependency faults and whether permitting bounded automated repairs is acceptable within their operational risk envelope. For contexts where initial generator variability or prompt engineering drives much of the failure mass, experimental designs that allow independent guarded first-pass generations or multi-sample selection are necessary to measure generator-side mitigation effects; those contrasts were intentionally out of scope here because of the preregistered `shared_initial_candidate` estimand.

## VI. LIMITATIONS

- Synthetic deterministic task bank and verifier: results reflect correctness under the chosen synthetic intent templates and deterministic verifier and do not guarantee similar performance on live heterogeneous production devices or telemetry.
- Single-model experiment (gpt-5-mini): findings do not demonstrate multi-model generality.
- `Shared_initial_candidate` semantics: baseline and guarded conditions began from the same initial generation; the study measures verifier/repair effects only and does not evaluate independent first-pass generation contrasts.
- Limited adversarial/contamination testing: the run did not include systematic adversarial retrieval or poisoned-context attacks; such threats remain an open evaluation axis.
- Oracle coverage: the deterministic verifier enforces encoded invariants but may omit runtime behaviors and device-specific semantics observable only through live execution; external validity to live deployments is therefore limited.

## VII. CONCLUSION

We executed a preregistered paired experiment measuring the marginal effect of a verifier-triggered bounded repair under `shared_initial_candidate` semantics for LLM-generated network configurations. In 200 synthetic deterministic tasks with a deterministic verifier and the declared hosted model gpt-5-mini (temperature recorded as null/unset), baseline acceptance was 199/200 and guarded acceptance was 200/200; one automated repair corrected an extreme multi-device ordering violation. Paired absolute risk difference = 0.005 with paired-bootstrap 95% CI [0.00, 0.015] (10,000 resamples, seed 1729) and exact two-sided McNemar  $p = 1.0$ . These artifact-grounded results support a constrained conclusion: verifier-guarded bounded repair can correct localized ordering faults in verifier-expressible tasks under the particular synthetic conditions studied. General claims about production readiness, multi-model benefits, independent guarded-generation effects, adversarial robustness, or dataplane-correctness require additional, distinct experiments (multi-model studies, independent-

first-pass guarded arms, adversarial contamination testing, and live-device/dataplane emulation). We provide full preregistration, execution accounting, verifier traces, and analysis artifacts in the archived execution bundle to support reanalysis and further controlled studies of guarded pipelines for NetOps automation.

## DISCLOSURE STATEMENT

Disclosure: This run implemented preregistered study CAND-2026-001 and was executed autonomously after the autonomy lock. Declared model: gpt-5-mini (model configuration recorded temperature = null/unset; null/unset is not evidence of deterministic sampling or temperature = 0). Orchestration adapter: `hosted_netops_gvr_v1`. Deterministic task generator: `deterministic_netops_task_generator_v5`. Deterministic verifier (oracle): `deterministic_netops_validator_v5`. Analysis executor: `paired_binary_analysis_v1`. Exact initial master prompt archived at `provenance/master_prompt.txt` (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Preregistration fixed `generation_semantics = shared_initial_candidate` (one sampled initial generation per task was deterministically reused by baseline and guarded episodes) and allowed at most one automated repair call per task. Model-call accounting reflects 200 shared initial generation calls and 1 repair call (201 model calls total). Humans selected the broad topic family and constrained the initial master prompt prior to the autonomy lock as permitted by the track; no human manually edited technical manuscript content after the autonomy lock. Archived provider traces, verifier logs, task manifests, model-configuration records, and analysis artifacts are available in the execution bundle for audit.

## REFERENCES

- [1] <https://doi.org/10.1145/3656296>
- [2] <https://doi.org/10.1145/3098822.3098834>
- [3] <https://doi.org/10.1038/s41591-024-03425-5>
- [4] <https://doi.org/10.69987/jacs.2024.40206>
- [5] <https://doi.org/10.1007/s11704-026-60308-3>
- [6] <https://doi.org/10.2139/ssrn.6647800>